

AI Search & Retrieval

Week 5 validation report

STATUS: COMPLETE

Natural-language policy search, confidence gating, and employee Q&A

Week 5 tested whether DeRexi can take an employee's natural-language question, locate the most relevant policy, return useful guidance, and avoid guessing when the policy library does not contain enough evidence.

01

Search

Natural-language queries return ranked policy matches.

02

Answer

Valid matches produce employee-facing guidance with a citation.

03

Guard

Unsupported questions are routed for clarification instead of guessed.

SEARCH TEST

Query: "What should an employee do if they receive a phishing email?"

The screenshot shows a web browser window displaying a REST client interface. The URL bar shows a GET request to `/policies/search`. The 'Parameters' tab is active, showing a query parameter `q` with the value `What should an employee do if they receive a phishing email?`. Below the query parameter, there is a 'limit' parameter set to `1`. The 'Responses' tab is also visible, showing the raw JSON response from the API. The 'Execute' button is highlighted in blue.

RESULT

Correct top match: Phishing & Social Engineering Awareness

Search relevance score: 0.274

The search endpoint successfully ranked the expected policy first.

CONFIDENCE GATE

0.22

Current cutoff for /ask

Scores run on a 0-to-1 similarity scale. The value is a relevance signal, not a percentage probability that the answer is correct.

What Testing Uncovered

A score above 0.22 was not enough by itself

Testing revealed a false-positive match caused by generic banking language.

DIAGNOSTIC EVIDENCE

Here's what I found. Current logic (canonical `backend/main.py`), verbatim:

Retrieval scoring (`SEARCH_SQL`, `main.py:68`):

```
... (0.6 * extensions.word_similarity(q, cv.body)
    + 0.4 * extensions.similarity(cv.title, :q)) AS score
```

Confidence gate (`/ask`, `main.py:333`): `confident = bool(results)` and `top_score >= 0.22`, then `answer = "Based on the {title} (v{no}): {excerpt}"`.

Measured behavior (live data, reproduced your numbers)

Question	Top policy	score	ws(body)	sim(title)	Top keyw. present in body/ title
"Does Aurum Capital Bank reimburse employees for gym memberships?"	Code of Conduct & Insider Integrity	0.2427	0.3973	0.0109	aurum✓ capital✓ bank✓ employees✓ · reimburse× memberships×
"What should I do if I receive a phishing email?"	Phishing & Social Engineering Awareness	0.2402	0.2826	0.1765	receive✓ phishing✓
"I think I got phished, how do I report it?"	Phishing & Social Engineering Awareness	0.2764	0.4000	0.0909	report✓ phished×

(`membership`, `gym`, `reimburse` appear in **zero** approved policies; `phishing/phish` in 2.)

Why the gym question outsourced the phishing one

`extensions.word_similarity(q, body)` is a **best-local-alignment** metric: it returns the best similarity between the query's trigram set and *any continuous window* of the policy text. It never requires the query's topic words to exist – a stretch where the generic tokens **Aurum / Capital / Bank / employees / for** all happen to appear (they're in nearly every policy) is enough to spike `ws_body` to 0.3973. The real signal in the phishing question is in the **title** (`sim_title=0.1765`, because "phishing" is in both), but title similarity only gets 40% weight, so

Why the first rule was not enough

The gym-membership question scored 0.2427 and initially matched Code of Conduct, even though the policy did not discuss reimbursements or memberships. Generic words such as bank and employees inflated similarity.

OPENCODE FIX

Proposed minimal fix (no schema change, touches only `/ask` + helpers)

- Substantive-keyword evidence gate.** After the existing `>= 0.22` check passes, require the top candidate to actually contain the question's substance:
 - `substantive_keywords(q) → tokens len>=4`, minus a curated `GENERIC_STOPWORDS` set (function words + policy, bank, aurum, capital, employees, staff, ...).
 - Gate: `any(kw in (title + " " + body).lower())` for `kw` in `substantive_keywords(q)`. Contains verifies whole/substring, so `reimburse` matches `reimbursement` and `memberships`→`membership` via a plural-strip fallback.
 - If the keyword set is empty, or no keyword is present → **not confident**.
- Route to clarification with your wording.** Both the low-confidence path and the new no-evidence path set `needs_clarification: true` with: "The policy library does not contain enough information to answer this confidently, so I have routed it to a policy owner for clarification rather than guess."
- Restructured answer for valid matches.** Pick the single most on-point complete sentence from the policy body (reusing `make_excerpt's` sentence selector) and compose guidance-first, citation-last:
Here's what you should do: (best sentence)
Reference: {Policy Title} (v{version.no}) – review the full policy for complete guidance.
(No LLM layer – it's the policy's own plain-language sentence, not the truncated raw excerpt; true paraphrasing would come later with an embeddings/LLM stage.)

Expected outcome on the observed cases: `gym` → keywords {`reimburse`, `memberships`}, neither in Code of Conduct → `needs_clarification: true`; `phishing` (both phrasings) → {`phishing`, `receive`} / {`phished`, `report`} present → confident answer citing the Phishing policy. `/policies/search`, `/dashboard`, and every other endpoint stay untouched.

Want me to implement it?

New rule: pass the 0.22 score AND show substantive keyword evidence before DeRexi answers.

Acceptance Tests

Three question patterns used to validate the final behavior

VALID ALTERNATE PHRASING

Request body required application/json

Edit Value Schema

```
{
  "user_id": 3,
  "question": "I think I got phished, how do I report it?",
  "topic": "phishing"
}
```

Execute Clear

UNSUPPORTED QUESTION

Parameters Cancel Reset

No parameters

Request body required application/json

Edit Value Schema

```
{
  "user_id": 3,
  "question": "Does Aurum Capital Bank reimburse employees for gym memberships?",
  "topic": "employee benefits"
}
```

TEST 1 - Original phishing question

"What should I do if I receive a phishing email?"

Initial confidence: 0.2402 | Correct policy retrieved

Finding: retrieval worked, but the first answer simply repeated a general excerpt. This test triggered the answer-quality improvement.

TEST 2 - Alternate phishing wording

"I think I got phished, how do I report it?"

Final confidence: 0.2764 | needs_clarification: false

Result: DeRexi returned actionable guidance - use the Report Phishing button or forward the message to the SOC, and do not click links, open attachments, or reply - followed by the policy citation.

TEST 3 - Unsupported gym reimbursement question

"Does Aurum Capital Bank reimburse employees for gym memberships?"

Before fix: 0.2427 and an incorrect Code of Conduct match

After fix: 0.2506 but blocked by evidence gate | needs_clarification: true

Final behavior: no citation was returned, DeRexi stated that the library lacked enough information, and the question was routed to a policy owner instead of guessed.

WEEK 5 OUTCOME

DeRexi now combines similarity scoring with substantive evidence, gives clearer guidance for valid matches, cites the supporting policy, and safely escalates unsupported questions.